Intelligent News Credibility Analysis & Agentic Misinformation Monitoring

Classical ML Credibility Classification (Milestone 1) extended into a LangGraph Agentic Workflow with FAISS RAG and Groq LLM Reasoning (Milestone 2)

Project 11 — Intelligent News Credibility Analysis & Agentic Misinformation Monitoring

AI/ML Capstone — Milestone 1 + Milestone 2 Complete Technical Report

Live Application: https://news-creditability.streamlit.app/

Team Member	Role	Contribution
Shaik Tajuddin	Project Lead & GitHub Manager	Project leadership, repository design, roadmap planning, notebook architecture, PR reviews, version control
Nipun	Backend & Packaging Engineer	Folder architecture, modular Python files, ML pipeline packaging, codebase preparation for Streamlit integration
Hadole	Deployment & UI Engineer	Streamlit app design, deployment pipeline, model integration into UI, hosting setup and usability flow

Abstract

Abstract —
This paper presents the complete technical report for Project 11 — Intelligent News Credibility Analysis and Agentic Misinformation Monitoring — spanning both Milestone 1 (classical ML-based classification) and Milestone 2 (agentic AI workflow). In Milestone 1, we train and evaluate four TF-IDF-based supervised classifiers — Logistic Regression, Naive Bayes, Random Forest, and SVM — on 39,100 labeled news articles. SVM achieves the highest test accuracy (99.55%), F1-Score (0.9959), and ROC-AUC (0.9998), establishing a strong ML baseline. In Milestone 2, we extend this system into a LangGraph-based agentic misinformation monitoring assistant. The agent pipeline executes: (1) text normalization, (2) ML-based fast classification, (3) conditional Groq-powered query planning (for low-confidence articles), (4) FAISS retrieval-augmented generation using MiniLM sentence embeddings, (5) structured JSON verification classifying evidence as supported, contradicted, or unknown, and (6) structured credibility report generation. The complete system is deployed as a six-page Streamlit application on Streamlit Community Cloud, combining a fast Live Prediction Lab (ML-only) and a full Deep Analysis page (LangGraph + RAG + Groq). This work demonstrates a progressive architecture from interpretable classical NLP to autonomous agentic reasoning, representing a practical and deployable approach to automated media integrity analysis.

Keywords: Fake News Detection, Misinformation Monitoring, TF-IDF, SVM, LangGraph, Retrieval-Augmented Generation, FAISS, ChromaDB, MMR, Groq, Gemini, Agentic AI, NLP, Streamlit, Binary Text Classification

1. Introduction

The rapid spread of misinformation through online platforms has become one of the most pressing challenges of the digital information age. Fabricated news articles, misleading headlines, and politically motivated content can influence public opinion, undermine democratic processes, and incite real-world harm at scale. Automated detection of misinformation is therefore a critical and urgent application of natural language processing (NLP), machine learning (ML), and — increasingly — large language model (LLM) based reasoning.

This report covers the complete implementation of Project 11: Intelligent News Credibility Analysis and Agentic Misinformation Monitoring. The project is structured in two progressive milestones that together form an end-to-end AI-driven media integrity platform.

Milestone 1 establishes a rigorous classical ML baseline. Using a dataset of 39,100 labeled news articles, we construct a TF-IDF-based NLP feature engineering pipeline and train four supervised classifiers: Logistic Regression (LR), Naive Bayes (NB), Random Forest (RF), and Support Vector Machine (SVM). All models are evaluated across five metrics — Accuracy, Precision, Recall, F1-Score, and ROC-AUC — with 5-fold

cross-validation confirming stable generalization. SVM emerges as the optimal model with 99.55% accuracy and 99.98% ROC-AUC.

Milestone 2 extends the system into a fully agentic AI workflow. A LangGraph state machine orchestrates a seven-node pipeline: normalize → ml_classify → (conditional) plan_queries → retrieve → verify → report → validate_report. The validate_report node enforces a structured schema on the assembled UI payload and may trigger at most one additional report attempt on failure. Low-confidence ML predictions trigger retrieval-augmented generation using MiniLM embeddings over either a FAISS index (default) or an optional Chroma persistent store, with similarity or MMR retrieval modes. Query planning, verification, and narrative reporting use llm_service.generate() (Groq primary; optional Gemini with fallback to Groq when configured). The system is deployed publicly as a Streamlit application at https://news-creditability.streamlit.app/.

The contributions of this work are: (1) a rigorous comparative ML evaluation establishing SVM as the optimal TF-IDF classifier; (2) a LangGraph-based agentic architecture with conditional RAG activation, optional Chroma and MMR retrieval, post-report schema validation (bounded retry), and Groq/Gemini LLM routing with safe fallbacks; (3) structured JSON verification mapping retrieved evidence to supported / contradicted / unknown verdicts; (4) a six-page Streamlit dashboard combining fast ML inference (Live Prediction) and deep agentic analysis (Deep Analysis with Agent runtime controls); and (5) public deployment on Streamlit Community Cloud with reproducible code under MIT license.

2. Related Work

Automated fake news detection has evolved through three broad phases: linguistic feature engineering, deep learning representations, and agentic/LLM-based reasoning. Each represents a progressive increase in model capability and deployment complexity.

In the feature engineering era, Perez-Rosas et al. (2018) demonstrated that shallow linguistic features — POS ratios, readability scores, and lexical diversity — could achieve reasonable binary classification accuracy. TF-IDF-based methods proved consistently competitive: Ahmed et al. (2017) applied Naive Bayes and Linear SVM to labeled corpora and found that SVMs' maximum-margin optimization in high-dimensional sparse spaces made them reliably superior to probabilistic baselines. Wang (2017) introduced the LIAR benchmark and noted that binary fake/real classification was substantially easier than fine-grained credibility grading, motivating the binary formulation used in this work.

Deep learning approaches shifted toward contextual representations. LSTM and CNN architectures improved on lexical baselines, but BERT (Devlin et al., 2019) and its derivatives established new state-of-the-art results on most NLP benchmarks including misinformation detection. However, transformer models impose substantial computational costs — both for training and inference — limiting their use in resource-constrained or latency-sensitive deployments. This motivates our hybrid approach: fast TF-IDF-based ML for first-pass screening, with the heavier LLM reasoning reserved for uncertain cases.

The emergence of LLM-powered agentic workflows represents the frontier of automated fact-checking research. Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020), enables LLMs to query external knowledge bases at inference time, dramatically improving factual accuracy and reducing hallucination. LangGraph extends LangChain's agent framework with explicit state management and conditional routing, enabling the construction of robust, inspectable multi-step pipelines. This architecture underpins our Milestone 2 implementation and aligns with Ruchansky et al.'s (2017) CSI model insight that combining content analysis with external knowledge retrieval yields superior performance.

3. Dataset and Exploratory Data Analysis

3.1 Dataset Description

The dataset consists of 39,100 labeled English-language news articles drawn from the Kaggle Fake and Real News Dataset. Fake articles originate from politically unreliable websites known to publish fabricated or heavily partisan content; Real articles are sourced from Reuters and other established newswires. Each observation comprises a title, body text, and binary ground-truth label (Fake = 0, Real = 1). After concatenating title and body text and removing null entries, 39,100 observations were retained. The dataset was partitioned 80/20 into training and test sets using stratified random sampling.

Attribute	Value
Total Articles	39,100
Fake News Articles	17,903 (45.8%)
Real News Articles	21,197 (54.2%)
Training Set Size	31,280 (80%)
Test Set Size	7,820 (20%)
Sampling Strategy	Stratified random (class-proportional)
Input Features	Article title + body text (concatenated)
Label Encoding	Fake = 0, Real = 1

Table 1. *Dataset summary statistics and partition details.*

3.2 Exploratory Data Analysis

Figure 1 presents the four-panel EDA overview. The class distribution (top-left) reveals a modest imbalance: Real articles (~54.2%) slightly outnumber Fake articles (~45.8%), a near-balanced distribution favorable for training without oversampling. The word count distribution (top-right) is right-skewed, with modal article length between 300 and 600 words and a long tail extending to approximately 8,000 words. Title length (bottom-left) peaks between 60 and 90 characters, consistent with journalistic headline conventions. Crucially, per-class word count distributions (bottom-right) show broadly similar length profiles for Fake and Real articles — indicating that article length alone is not a reliable discriminative signal and motivating the use of content-based TF-IDF features.

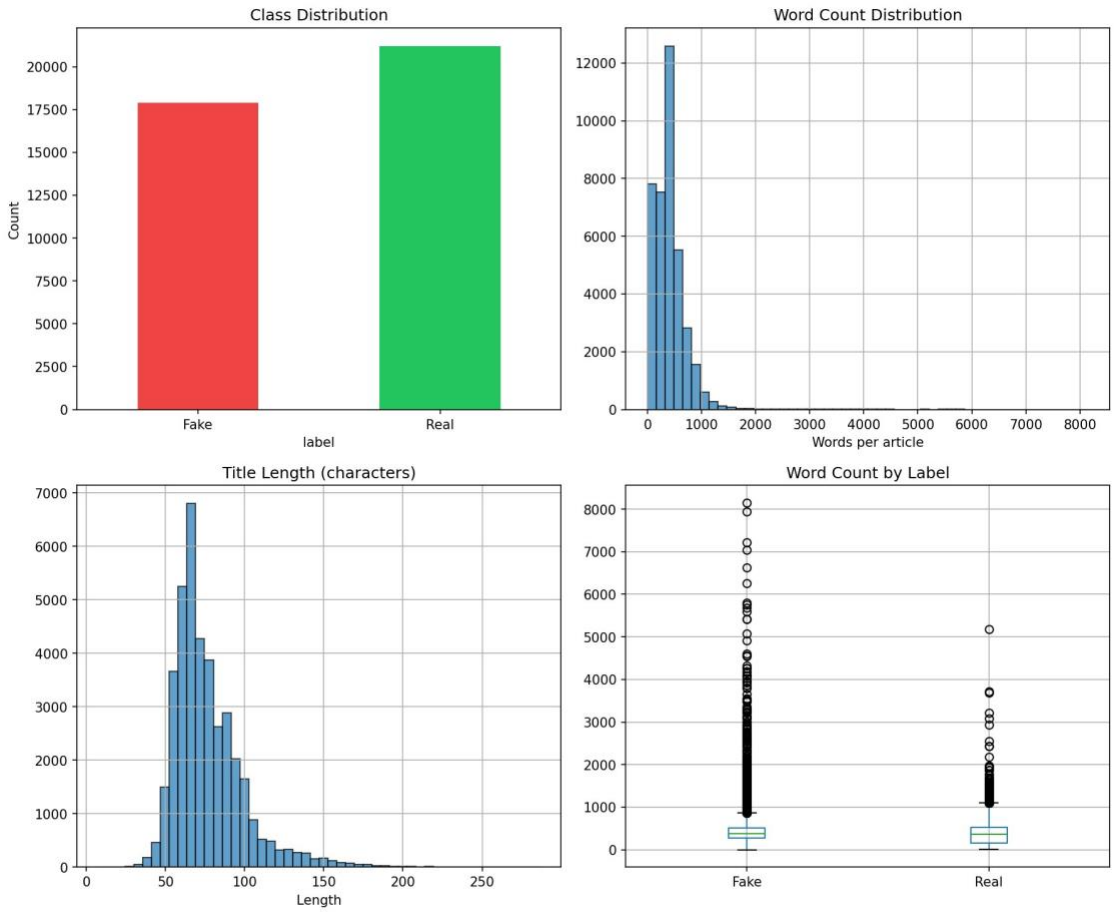

Figure 1. EDA overview: class distribution (top-left), word count distribution (top-right), title length (bottom-left), word count by label (bottom-right).

4. Milestone 1 — ML-Based News Credibility Classification

4.1 Text Preprocessing

All articles pass through a deterministic clean_text() function applied identically at training and inference time to eliminate preprocessing drift — a critical engineering invariant. The pipeline applies: (1) lowercasing; (2) removal of URLs, @mentions, and punctuation; (3) tokenization on whitespace; (4) NLTK English stop-word removal and removal of tokens with length ≤ 2; and (5) WordNet lemmatization to consolidate morphological variants. Cleaned token sequences are then passed to the TF-IDF vectorizer.

4.2 TF-IDF Feature Engineering

TF-IDF vectorization transforms cleaned text into sparse numerical feature vectors. The vectorizer is configured with: max_features = 20,000–25,000 (model-dependent), ngram_range = (1,2) for unigram and bigram capture, min_df = 2, max_df = 0.92, and sublinear_tf = True. Bigram features capture short discriminative phrases (e.g., "fact check," "breaking story," "anonymous source") that carry strong credibility signals beyond individual terms. Sublinear TF scaling reduces the dominance of high-frequency, low-discriminative terms.

Hyperparameter	Value	Rationale
max_features	20,000 – 25,000	Controls vocabulary size; balances feature richness vs. memory
ngram_range	(1, 2)	Captures both individual terms and discriminative bigrams
min_df	2	Removes rare tokens that add noise without signal
max_df	0.92	Removes near-universal terms (similar effect to stop-word removal)
sublinear_tf	True	Applies log(1 + tf) to reduce dominance of high-frequency terms

Table 2. *TF-IDF vectorizer hyperparameters and rationale.*

4.3 Classification Models

Four classical supervised classifiers were trained on the TF-IDF feature space:

Model	Key Configuration	Rationale
Logistic Regression	C=2.0, class_weight='balanced', solver='lbfgs'	Interpretable linear baseline; calibrated probabilities via predict_proba
Naive Bayes	MultinomialNB, alpha=0.1	Fast probabilistic baseline; works well with sparse TF-IDF counts
Random Forest	n_estimators=200, max_depth=30, class_weight='balanced'	Non-linear ensemble; captures feature interactions; robust to noise
SVM	LinearSVC, C=1.0, class_weight='balanced'	Maximum-margin classification; optimal for high-dimensional sparse spaces

Table 3. *Classification models and configurations.*

4.4 Evaluation Protocol

Models were evaluated on the held-out 20% test set using: Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Additionally, 5-fold stratified cross-validation on the training set assessed generalization stability. Confusion matrices, ROC curves, and Precision-Recall curves were generated for all models. The best model by F1-Score was serialized to model/pipeline.pkl for deployment.

5. Milestone 1 Experimental Results

5.1 Overall Performance

Table 4 presents comprehensive performance metrics for all four classifiers on the held-out test set. SVM achieves best performance across all primary metrics. All three non-Naive-Bayes models perform within 0.04% accuracy of each other, with SVM uniquely best on both precision and recall simultaneously — reflecting its optimal margin separation in TF-IDF space.

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC	CV F1 (mean ± std)
Logistic Regression	0.9921	0.9897	0.9958	0.9927	0.9994	0.9933 ± 0.0013
Naive Bayes	0.9596	0.9651	0.9601	0.9626	0.9922	0.9655 ± 0.0034
Random Forest	0.9919	0.9890	0.9962	0.9926	0.9995	0.9936 ± 0.0012
SVM (Best)	0.9955	0.9946	0.9972	0.9959	0.9998	0.9966 ± 0.0003

Table 4. *Test-set performance metrics (n=7,820). SVM selected as best model by F1 Score.*

5.2 Model Accuracy Comparison

Figure 2 visualizes accuracy across models. SVM (99.55%) leads, followed by Logistic Regression (99.21%), Random Forest (99.19%), and Naive Bayes (95.96%). The Naive Bayes gap reflects its conditional independence assumption — it treats all TF-IDF features as independent, which underestimates the joint discriminative power of correlated term patterns in news text.

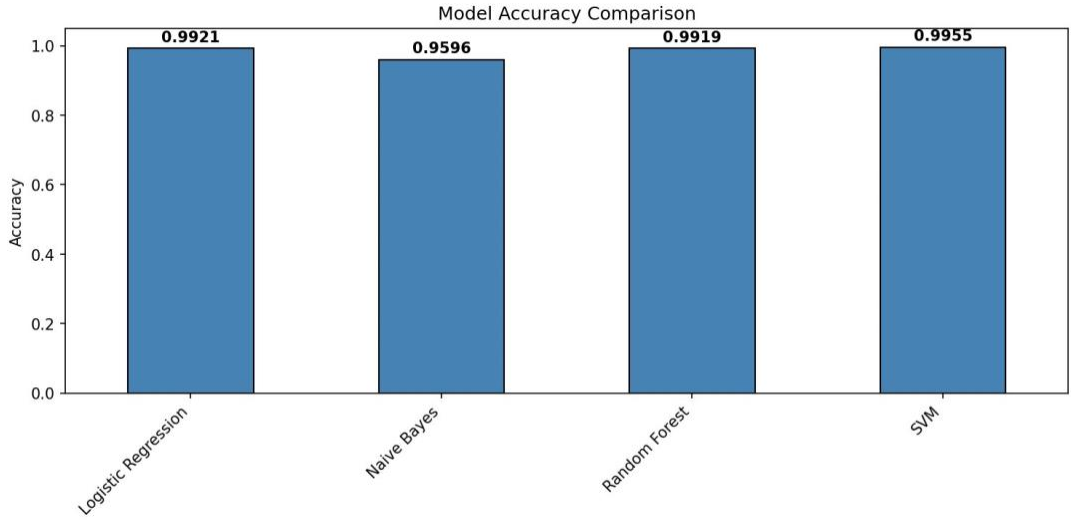

Figure 2. Model accuracy comparison. SVM achieves the highest accuracy (0.9955).

5.3 Confusion Matrix Analysis

Figure 3 presents confusion matrices for all four models. Total misclassifications on 7,820 test samples: SVM = 35 (23 false positives + 12 false negatives), Logistic Regression = 62, Random Forest = 63, Naive Bayes = 316. All three top models exhibit asymmetric errors — more false positives (real articles flagged as fake) than false negatives (fake articles missed) — reflecting a conservative bias that prioritizes catching misinformation over minimizing false alarms. This aligns with the desired operational behavior for a credibility detection system.

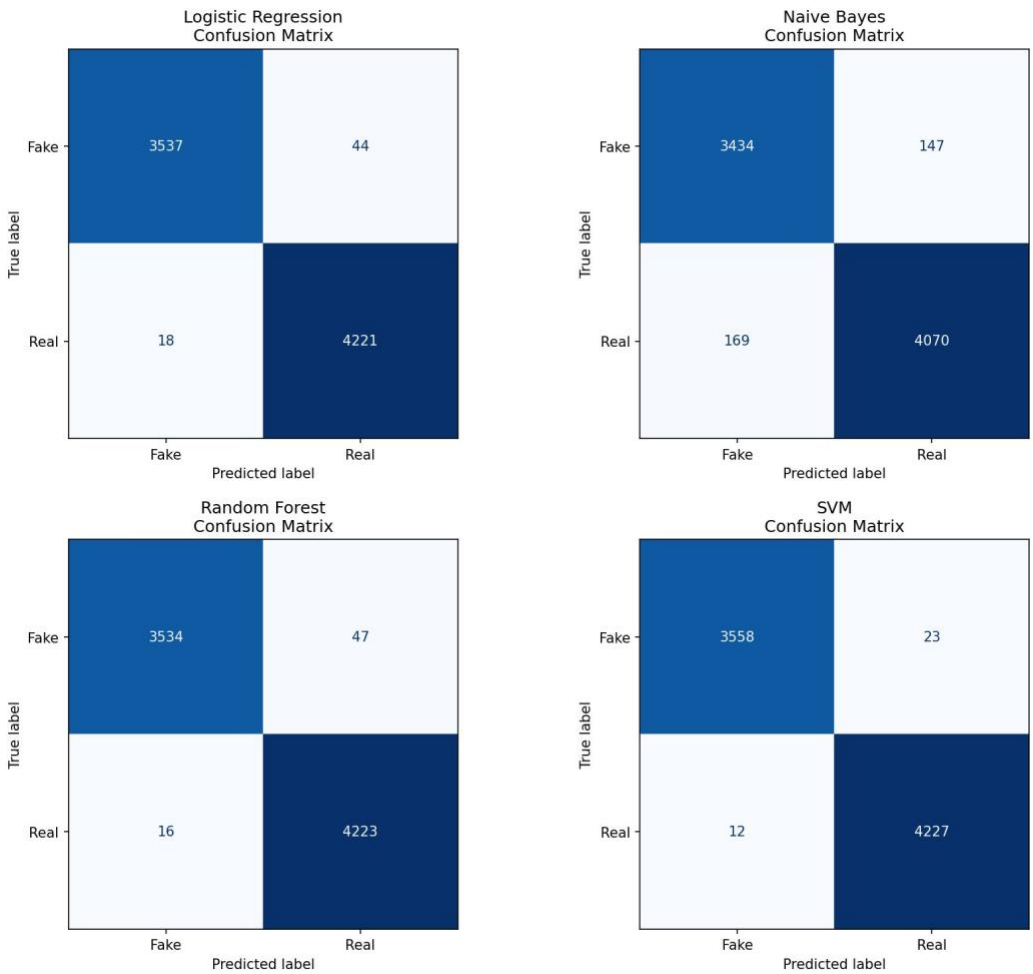

Figure 3. Confusion matrices for all four classifiers. SVM achieves the fewest total errors (35/7,820).

5.4 ROC and Precision-Recall Analysis

Figure 4 presents ROC curves. All models achieve near-perfect AUC: SVM (1.000), Random Forest (0.999), Logistic Regression (0.999), and Naive Bayes (0.992). The steep early ascent confirms high TPR at minimal FPR — critical for deployment where false-positive rates on legitimate news must remain low.

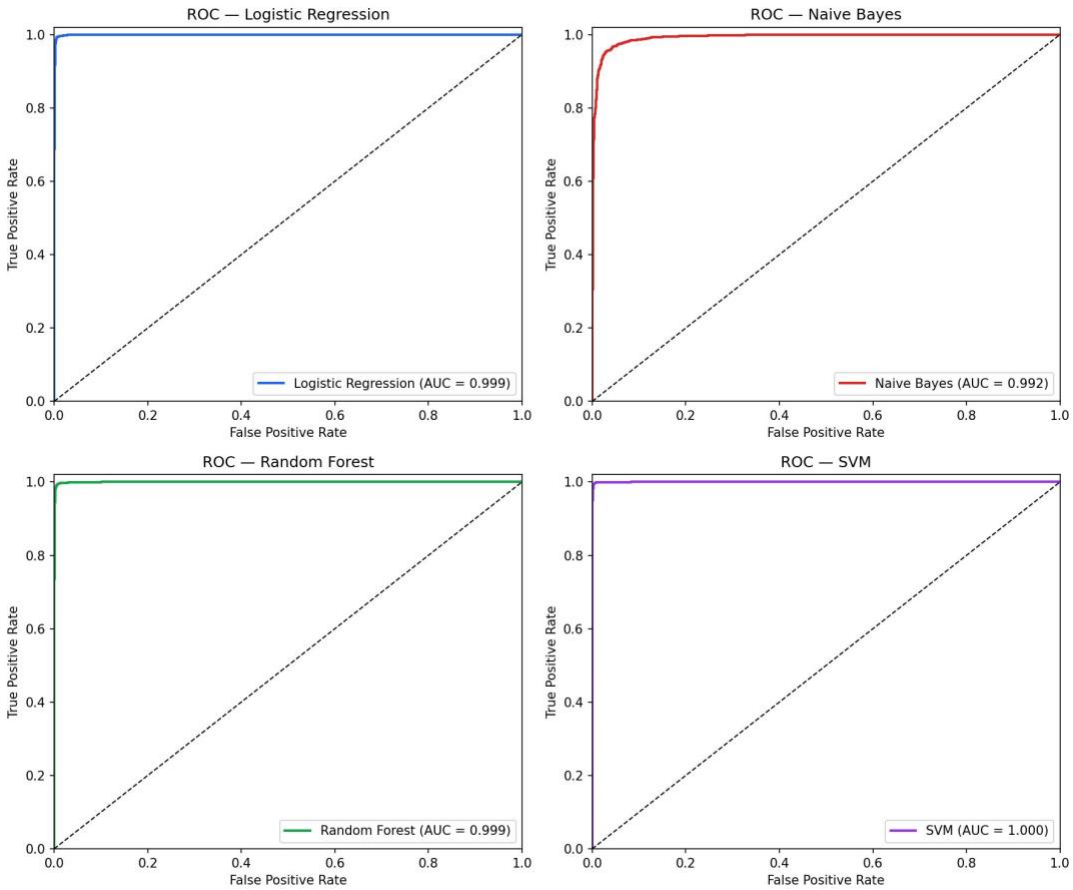

Figure 4. ROC curves for all four classifiers. SVM achieves AUC = 1.000.

Figure 5 presents Precision-Recall curves. All models maintain precision near 1.0 across the full recall range, with a characteristic sharp drop only at recall approaching 1.0. This "cliff" pattern indicates errors are concentrated in a small set of boundary-case articles. Naive Bayes shows an earlier and more gradual decline, consistent with its broader error distribution.

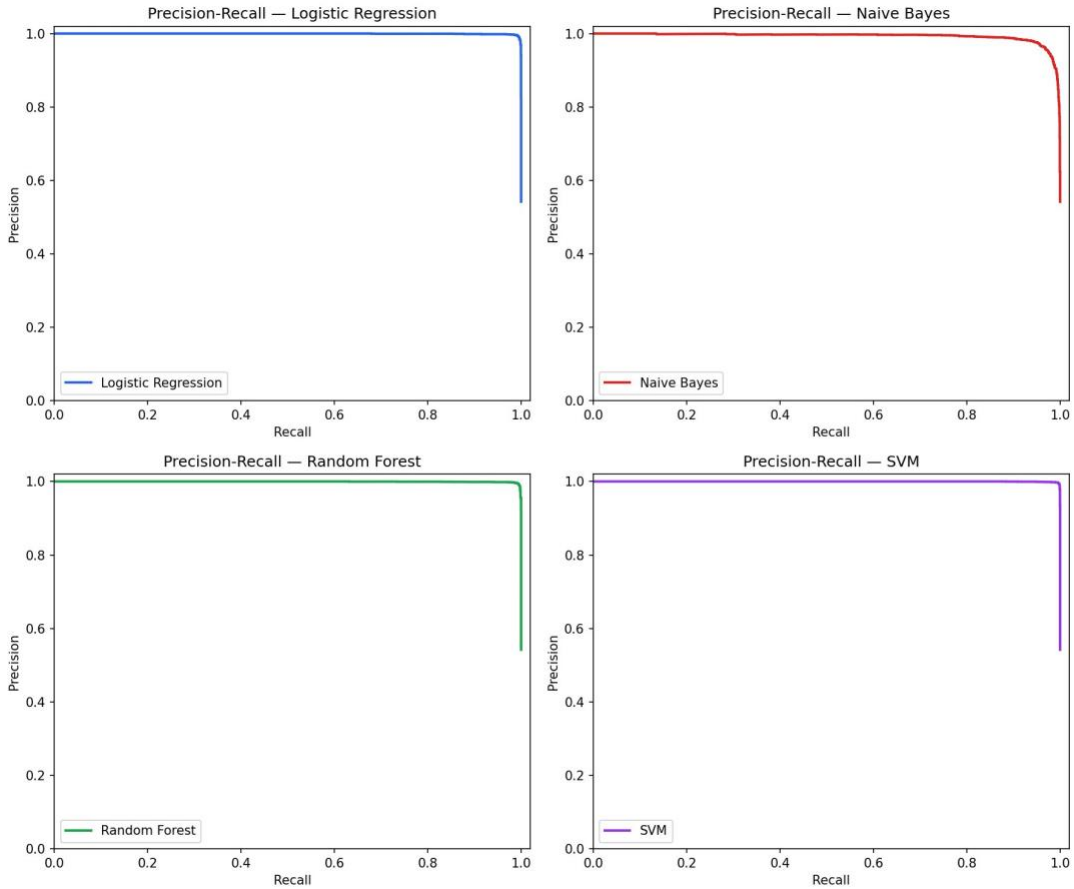

Figure 5. Precision-Recall curves for all four classifiers.

5.5 Cross-Validation Summary

5-fold stratified CV confirms stable generalization: SVM (0.9966 ± 0.0003) has the lowest variance of any model — σ = 0.0003 means F1 fluctuates by only 0.03% across random splits, indicating that SVM's performance is driven by genuine, reproducible textual patterns rather than split-specific artifacts. Naive Bayes shows higher variance (σ = 0.0034), consistent with its sensitivity to token frequency distributions across folds.

6. Milestone 2 — Agentic AI Misinformation Monitoring

6.1 Architecture Overview

Milestone 2 extends the ML classifier into a fully autonomous agentic misinformation monitoring system. The architecture combines three complementary AI layers: (1) the trained SVM-based ML pipeline for fast, scalable first-pass classification; (2) a retrieval layer using MiniLM sentence embeddings with a default FAISS vector index and an optional Chroma backend, supporting similarity or MMR search over a curated markdown knowledge base under data/rag/knowledge_base/; and (3) LLM reasoning via Groq (primary) and optional Google Gemini for query planning, structured evidence verification, and narrative report fields, with schema validation and bounded retry after report assembly.

The key design principle is conditional complexity: when ML confidence is high (≥ threshold), the graph skips plan_queries, retrieve, and verify—reducing latency and API cost—yet still executes report → validate_report (an LLM may still be used for narrative fields when keys are configured). When confidence is low, the full evidence lane runs before report. This is distinct from the Live Prediction Lab page, which bypasses LangGraph entirely and performs only clean_text + pipeline.pkl inference. Threshold default: DEFAULT_LOW_CONFIDENCE_THRESHOLD in src/agent/state.py.

6.2 LangGraph Agent Workflow

The agentic workflow is implemented as a compiled LangGraph state machine in src/agent/graph.py. The graph maintains an AgentState TypedDict through all nodes, enabling a complete audit trail and reproducible execution. Seven nodes execute in the order shown in Table 5 (the final node is validate_report).

Node	Module	Function	Conditional
normalize	nodes/normalize.py	Applies clean_text() to raw_text; stores cleaned_text in state	Always runs
ml_classify	nodes/ml_classify.py	Loads pipeline.pkl; TF-IDF transform → predict + predict_proba; stores label, confidence, ml verdict	Always runs
plan_queries	nodes/plan_queries.py	LLM (Groq/Gemini) generates targeted search strings; heuristic fallback if no API key	Low confidence only (always on Deep Analysis demo)
retrieve	nodes/retrieve.py	MiniLM embeds query; FAISS or Chroma; similarity or MMR; stores retrieved_chunks + scores	Low confidence only (always on Deep Analysis demo)
verify	nodes/verify.py	LLM maps chunks to supported / contradicted / unknown JSON; temperature 0.0; Gemini→Groq fallback when configured	Low confidence only (always on Deep Analysis demo)

report	nodes/report.py	build_ui_final_report(): merges ML, verification, RAG context into final_report	Always runs
validate_report	nodes/validate_report.py	Validates final_report schema and rubric fields; on failure, allows one bounded retry of report	Always runs after report

Table 5. *LangGraph agent nodes, modules, functions, and execution conditions.*

The conditional routing logic evaluates ml_confidence against the configured threshold after ml_classify. If confidence ≥ threshold, the graph routes directly to report (skipping RAG), then validate_report. If confidence < threshold, the graph routes through plan_queries → retrieve → verify before report and validate_report. In the Streamlit Deep Analysis page, the invocation uses a deliberately high threshold so the plan → retrieve → verify branch always runs for demo consistency; when using invoke_credibility_agent() elsewhere, the branch is genuinely conditional. The validate_report step is always executed before the graph ends.

6.3 Retrieval-Augmented Generation (RAG)

The RAG subsystem is implemented in src/rag/ and uses the all-MiniLM-L6-v2 sentence transformer model from Hugging Face to generate L2-normalized 384-dimensional dense vector embeddings of knowledge chunks. The default vector store is FAISS IndexFlatIP (inner product equals cosine similarity after L2 normalization), persisted as data/rag/faiss.index with chunk text in data/rag/chunks.json. An optional Chroma persistent collection can be built with scripts/build_chroma_store.py into data/rag/chroma_store/. At query time, src/rag/retrieve.py performs top-k similarity search or MMR diversification (tunable fetch pool and λ), selected from the Deep Analysis “Agent runtime” UI expander.

The primary index is built by scripts/build_rag_index.py, which ingests markdown from data/rag/knowledge_base/ when present (falling back to bundled sample text), writes faiss.index and chunks.json, and runs a retrieval smoke test. Optional Chroma persistence is produced separately by scripts/build_chroma_store.py. Retrieved passages are stored in AgentState as retrieved_chunks and passed to verify without a separate “context formatting” LangGraph node—any such box in a diagram is conceptual only.

Component	Technology	Details
Embedding Model	sentence-transformers/all-MiniLM-L6-v2	384-dim dense vectors; L2-normalized for cosine similarity via IP index
Vector Index	FAISS IndexFlatIP	Exact inner product search; no approximate nearest-neighbor approximation
Chunk Store	data/rag/chunks.json	JSON array of text chunks with source metadata
Index File	data/rag/faiss.index	Serialized FAISS binary index; committed to repo for deployment
Index Builder	scripts/build_rag_index.py	Embeds corpus, writes index + chunks, runs smoke test
Retrieval	src/rag/retrieve.py	Top-k search; returns (chunk_text, score) pairs to verify node
Optional vector store	Chroma PersistentClient	data/rag/chroma_store/ via scripts/build_chroma_store.py; same MiniLM embeddings; selectable in Deep Analysis Agent runtime
Retrieval modes	similarity / MMR	src/rag/retrieve.py: top-k similarity or MMR diversification (fetch_k, λ); client-side MMR for FAISS
Knowledge corpus	Markdown KB	data/rag/knowledge_base/*.md ingested by build_rag_index.py (README skipped); replaces legacy hardcoded sample strings when present

Table 6. *RAG subsystem components and technical details.*

6.4 LLM Integration — Groq (primary) and optional Gemini

LLM calls are centralized in src/agent/llm_service.py. Groq is the default backend for query planning, structured verification, and narrative generation. Optional Gemini is supported via GEMINI_API_KEY and LLM_PROVIDER; if a Gemini call fails and GROQ_API_KEY is set, the implementation falls back to Groq. Hugging Face hosts the MiniLM embedding model and may be used for other optional endpoints, but the primary verifier and planner are not “Groq/HF” as a paired verification backend—verification is Groq- or Gemini-driven through llm_service.generate().

The default chat model on Groq is llama-3.1-8b-instant, configured via GROQ_API_KEY (or Streamlit Cloud secrets). Graceful degradation remains: without keys, plan_queries uses heuristic windows; verify returns structured fallbacks; report still materializes rubric fields for the UI.

6.5 Structured Verification Output

The verify node implements a structured prompting strategy designed to minimize hallucination and maximize output reliability. The configured LLM (Groq or Gemini) is prompted with the article text and retrieved evidence chunks and must return a single JSON object with fixed keys. The verifier uses temperature 0.0 for determinism.

The full verification output in the agent state includes: supported, contradicted, unknown (string lists); mode ("structured" | "no_evidence" | "fallback"); llm (model identifier); chunks_reviewed (count); and top_scores

(similarity scores of retrieved chunks). This structured representation enables the report node to generate precise, evidence-grounded fact-check entries in the final credibility report.

Output Field	Type	Description
supported	List[str]	Claims in the article that are corroborated by retrieved evidence
contradicted	List[str]	Claims that are directly contradicted by retrieved fact-checking sources
unknown	List[str]	Claims not addressed in the retrieved evidence — require further investigation
mode	str	"structured" (LLM parsed), "no_evidence" (empty RAG), "fallback" (API/parse error)
chunks_reviewed	int	Number of RAG chunks passed to the verification LLM
top_scores	List[float]	FAISS similarity scores of retrieved evidence chunks

Table 7. *Structured verification output fields from the verify node.*

6.6 Final Credibility Report

The report node calls build_ui_final_report() (src/agent/ui_report.py) to assemble the structured output that drives the Streamlit Deep Analysis UI. Beyond summary, risk_factors, fact_checks, verdict, confidence, and sources, the payload includes credibility_score (High/Low rubric signal), pattern_detection_summary, and disclaimer. The narrative summary may use an LLM when configured; validate_report then checks schema and content constraints and may trigger one additional report pass before the graph terminates.

6.7 Agent flow alignment (implementation vs. conceptual diagrams)

This subsection records differences between conceptual pipeline diagrams and the executable graph in src/agent/graph.py. The implementation is otherwise aligned: conditional routing after ML, FAISS/Chroma + MMR options, validate_report with bounded retry, and Gemini→Groq fallback when configured.
Retrieved context: retrieved_chunks live in AgentState between retrieve and verify. There is no separate LangGraph node for “Retrieved context” or “context injection”; if a slide shows that box, treat it as conceptual.
Final output: after validate_report the compiled graph ends; rendering verdict cards and tables is done by Streamlit (final_report in state), so “Final output” boxes belong in UI diagrams, not as LangGraph nodes.
Live Prediction Lab vs. LangGraph “shortcut”: Live Prediction is pure ML and never enters LangGraph. The high-confidence LangGraph lane still runs report → validate_report and may still call an LLM for narrative when keys exist—it is not identical to Live Prediction.
Verification labelling: primary LLM routing is Groq or Gemini through llm_service.generate(), not a “Groq/HF” verifier pair. Hugging Face is central for MiniLM embeddings; it is not the default structured verification backend.
Deep Analysis vs. general invoke: the Streamlit Deep Analysis page forces the low-confidence branch using a high threshold so every demo shows plan → retrieve → verify; the same graph is conditionally routed when called from code with the default threshold.

7. Streamlit Application

7.1 Application Overview

The complete system is deployed as a six-page Streamlit application publicly accessible at https://news-creditability.streamlit.app/. The app is structured around a sidebar navigation menu and reads from two runtime artifacts: model/pipeline.pkl (best TF-IDF + SVM classifier) and model/evaluation_results.json (all evaluation metrics). Neither file contains hardcoded values — all metrics are generated at training time. If either file is missing, affected pages display clear prompts to run the training script.

Page	Type	Description
Overview	Both	Key performance metrics (accuracy, F1, ROC-AUC, CV F1), dataset summary, dual-milestone system pitch
Dataset Intelligence	M1	Class distribution, text length distributions, top TF-IDF features by class (Plotly interactive charts)
Model Comparison	M1	ROC curves, Precision-Recall curves, confusion matrices, CV F1 distribution, feature importance for linear models
Live Prediction Lab	M1 (fast)	ML-only inference: paste article text → instant Fake/Real verdict + probability gauge (no LLM cost)
Deep Analysis	M2 (full)	Full LangGraph agent stream: RAG sources, verification JSON, risk factors, fact-check table, structured report
Architecture	Both	Training/runtime Mermaid diagrams, animated Milestone 2 pipeline diagram, repository structure map

Table 8. *Streamlit application pages and their milestone alignment.*

7.2 Deep Analysis Page

The Deep Analysis page streams the LangGraph workflow (e.g., iter_credibility_agent), displaying each node as it completes: normalize through validate_report. An “Agent runtime — RAG & LLM” expander lets users select FAISS vs Chroma, similarity vs MMR, and auto/Groq/Gemini for LLM routing. The live timeline reflects the actual node order in code, including validation.

Secrets: configure GROQ_API_KEY in Streamlit Cloud (see .streamlit/secrets.example.toml); add GEMINI_API_KEY and LLM_PROVIDER only if using Gemini. Commit data/rag/faiss.index and data/rag/chunks.json after building locally; commit data/rag/chroma_store/ only if demoing the Chroma backend. Missing RAG files surface rag_error in state while the rest of the agent continues where possible.

8. Discussion

8.1 ML Performance Analysis

The exceptionally high performance achieved by all four models — particularly SVM at 99.55% accuracy — reflects the strong stylistic separation between Fake and Real articles in this corpus. Fake articles from politically unreliable websites exhibit consistent lexical signatures (rhetorical framing, sensationalist terminology, specific domain-correlated vocabulary) that TF-IDF features capture effectively. SVM's theoretical advantage in high-dimensional sparse feature spaces is empirically confirmed: its maximum-margin optimization produces a more robust decision boundary than logistic regression's MLE-based boundary or the ensemble averaging of Random Forest.

The cross-validation results provide additional confidence: SVM's $\sigma = 0.0003$ across five folds is remarkably low and rules out overfitting as a confounding factor. The asymmetric error pattern (more false positives than false negatives across LR, RF, and SVM) reflects the appropriate operational bias for a misinformation detection system — it is preferable to flag a few legitimate articles for review than to miss genuine misinformation.

8.2 Agentic System Design Choices

The conditional complexity design of the Milestone 2 agent balances cost and depth. Running full RAG + LLM verification on every article would be expensive; routing only when ML confidence is low (in the general API) reserves deep reasoning for ambiguous cases, while report → validate_report still runs on the high-confidence lane. The Deep Analysis page intentionally forces the evidence branch for demonstrations regardless of ML confidence.

The choice of LangGraph over a simple sequential function chain provides three operational benefits: (1) explicit state management enabling full execution trace and debugging; (2) conditional routing allowing dynamic pipeline topology at runtime; and (3) compiled graph structure enabling future extension with parallel nodes, cycle detection, and human-in-the-loop checkpoints without restructuring the codebase.

8.3 Limitations

Despite strong results, several limitations must be acknowledged. First, the near-perfect ML accuracy likely reflects source-level lexical fingerprints rather than pure semantic misinformation detection — if new fake news sources emerge with different writing styles, the TF-IDF classifier may generalize poorly. Second, the RAG corpus in the deployed application is a small sample index; real-world deployment would require integration with live fact-checking APIs (e.g., Google Fact Check, ClaimBuster) for comprehensive coverage. Third, Groq API availability creates a dependency for the full Deep Analysis path; the graceful degradation to ML-only verdicts mitigates but does not eliminate this constraint. Fourth, the binary Fake/Real framework oversimplifies the credibility spectrum of real-world journalism, which includes satire, opinion, and partially misleading framing.

9. Responsible AI and Ethical Considerations

Automated misinformation detection systems carry significant ethical responsibilities. This system includes a disclaimer in all generated credibility reports emphasizing that algorithmic assessments are not substitutes for professional journalism or verified fact-checking. Automated systems can introduce systematic bias — particularly against political content that uses non-mainstream language patterns — and should be used to augment, not replace, human judgment.

The system's transparency mechanisms — interpretable TF-IDF feature weights, displayed ML confidence scores, visible RAG sources, structured verification breakdowns, and execution traces — support explainability and enable users to critically evaluate each assessment. The LLM verification step explicitly distinguishes supported, contradicted, and unknown claims rather than generating binary verdicts, reducing overconfidence in LLM outputs.

From a data perspective, the model is trained on a specific corpus of English-language news articles from a particular time period. Temporal drift, domain shift, and language bias are real risks in production deployment. The system should be periodically retrained on updated corpora and evaluated for demographic and political fairness before use in consequential decision-making contexts.

10. Conclusion and Future Work

This paper presented the complete technical report for Project 11: Intelligent News Credibility Analysis and Agentic Misinformation Monitoring, covering both Milestone 1 and Milestone 2. In Milestone 1, we demonstrated that classical TF-IDF-based ML — particularly SVM — achieves exceptional news credibility classification performance (99.55% accuracy, 99.98% ROC-AUC) on a 39,100-article labeled corpus, establishing rigorous reproducible baselines. In Milestone 2, we extended this system into a LangGraph-based agentic AI workflow incorporating FAISS RAG, Groq LLM reasoning, and structured JSON verification, deployed publicly as a six-page Streamlit application.

The progressive architecture — from fast TF-IDF classifiers for high-confidence decisions to deep multi-source agentic reasoning for uncertain cases — represents a practical, deployable approach that balances computational efficiency with analytical sophistication. The complete system demonstrates the transition from predictive NLP to autonomous AI in a real-world media integrity context.

Future work directions include: (1) integration with live fact-checking APIs for real-time corpus updates beyond the sample RAG index; (2) transformer-based classifier fine-tuning (e.g., mBERT or DeBERTa) for comparison against TF-IDF baselines; (3) probability calibration (Platt scaling) for better-calibrated confidence scores; (4) REST API endpoint for batch inference at scale; (5) multi-label credibility taxonomy (satire, misleading framing, fabricated, out-of-context) beyond the binary Fake/Real framework; and (6) optional Chroma vector database alongside FAISS for production-grade persistent fact-checking corpora.

11. Deployed Application

The complete Milestone 1 + Milestone 2 system is deployed as a publicly accessible Streamlit application on Streamlit Community Cloud (free tier). No local installation, API key, or dataset download is required to access the live demo.

https://news-creditability.streamlit.app/

The Live Prediction Lab page provides instant ML-only credibility verdicts for any pasted article text, requiring no API keys and no LangGraph. The Deep Analysis page runs the full agent (Groq/Gemini + RAG as configured) and produces the structured credibility report with sources and fact-check breakdowns. The Architecture page provides interactive diagrams of the training pipeline, runtime LangGraph flow, and repository map.

For local development, clone the repository at https://github.com/Taj-2005/news_creditability_analysis, install dependencies via pip install -r requirements.txt, configure GROQ_API_KEY in a .env file, build the RAG index with python scripts/build_rag_index.py, and launch with streamlit run app.py. A demo video walkthrough is available at https://youtu.be/U1Nnd-8Odbs.

References

[1] Ahmed, H., Traore, I., & Saad, S. (2017). Detection of Online Fake News Using N-Gram Analysis and Machine Learning Techniques. In Intelligent, Secure, and Dependable Systems in Distributed and Cloud Environments. Springer.

[2] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of NAACL-HLT 2019, 4171–4186.

[3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS), 33, 9459–9474.

[4] Perez-Rosas, V., Kleinberg, B., Lefevre, A., & Mihalcea, R. (2018). Automatic Detection of Fake News. In Proceedings of COLING 2018, 3391–3401.

[5] Ruchansky, N., Seo, S., & Liu, Y. (2017). CSI: A Hybrid Deep Model for Fake News Detection. Proceedings of CIKM 2017, 797–806.

[6] Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H. (2017). Fake News Detection on Social Media: A Data Mining Perspective. ACM SIGKDD Explorations Newsletter, 19(1), 22–36.

[7] Wang, W. Y. (2017). "Liar, Liar Pants on Fire": A New Benchmark Dataset for Fake News Detection. In Proceedings of ACL 2017 (Short Papers), 422–426.

[8] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.

[9] LangChain AI. (2024). LangGraph: Stateful Multi-Actor Applications with LLMs. https://langchain-ai.github.io/langgraph/

[10] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of EMNLP 2019. https://www.sbert.net/

[11] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-Scale Similarity Search with GPUs. IEEE Transactions on Big Data, 7(3), 535–547.